

Personal Reference Document — Confidential

AI RESUME ANALYZER

Project Builder's Reference

Resume Entry · How You Built It · Skills Demonstrated · Interview Prep

Prepared For

Anik Dhiman

3rd Year B.Tech CSE (AI/ML) — JSS AITE, Noida

[GitHub: TheAnikDhiman](#)

PART 1 — YOUR RESUME ENTRY

Copy-paste this directly into your resume under the Projects section.

Resume Entry — Copy Exactly:

AI Resume Analyzer — ATS Score Checker Using NLP FastAPI · React (Vite) · Python · Tailwind CSS · pdfplumber · NLP · ReportLab · Git • Built end-to-end NLP pipeline for automated resume analysis — PDF text extraction, 40+ skill detection, ATS scoring, and JD-specific match scoring • Architected REST API using FastAPI (3 endpoints) serving React + Tailwind CSS frontend with real-time analysis, animated score gauges, and streaming PDF download • Implemented keyword-based NLP skill extraction, rule-based suggestion engine, and resume section validator as independent modular Python functions • Migrated from Streamlit prototype to production-grade React + FastAPI full-stack architecture, improving UI quality and enabling component-based frontend development • Generated downloadable ATS reports using ReportLab with in-memory BytesIO streaming — no disk I/O required

Resume Bullet Variations

Use these variations depending on the role you are applying for:

For ML/AI Intern Roles:

- Built NLP pipeline for automated resume analysis using keyword extraction, TF-IDF-style scoring, and rule-based intelligence — served via FastAPI REST API
- Designed modular Python architecture with 6 independent NLP processing functions, enabling clean separation of text extraction, skill detection, scoring, and suggestion logic

For Full-Stack / Backend Intern Roles:

- Architected full-stack web application with FastAPI backend (Python) and React frontend — implemented file upload, real-time JSON API, CORS, and streaming PDF response
- Built React component architecture with hooks for state management, Recharts for data visualization, and Axios for API communication with FastAPI backend

For General SWE Roles:

- Developed full-stack AI Resume Analyzer from prototype to production — Streamlit MVP → React + FastAPI architecture with PDF processing, NLP scoring, and report generation

PART 2 — HOW YOU BUILT IT (For Interviews)

This is your story. Know this well — interviewers love to ask 'walk me through how you built this.'

The Build Story — Tell It Like This

Opening:

I built this project to help a friend who needed a BCA final year major project. I started with a Streamlit prototype to validate the core idea quickly, then migrated to a proper full-stack architecture with React and FastAPI once the logic was proven.

Phase-by-Phase Breakdown

Phase	What You Built & Why
Phase 1 — Prototype (25%)	Streamlit app with PDF upload, pdfplumber extraction, keyword matching, basic score display. Goal: prove the core pipeline works before investing in full-stack.
Phase 2 — Features (50%)	Added JD matching module, Plotly gauges, section checker. Key insight: comparing against JD is more valuable than a generic score.
Phase 3 — Expansion (70%)	Grew skill list from 16 to 40+ skills across 6 categories. Improved UI with color-coded results.
Phase 4 — Full Stack (100%)	Migrated to React + FastAPI. Built drag-drop upload with react-dropzone, RadialBarChart gauges with Recharts, skill chips, PDF report streaming with ReportLab.

Key Technical Decisions You Made

Decision 1: Why FastAPI over Flask?

Flask was familiar but FastAPI gives async support, automatic Swagger docs, and Python type hints — which is the modern production standard. For a system that will eventually serve concurrent users, async matters. I also wanted to learn industry-standard tools, not just comfortable ones.

Decision 2: Why keyword matching over spaCy?

spaCy's NER models are not trained on technology brand names like 'FastAPI', 'PyTorch', or 'Flutter'. Case-insensitive keyword matching against a curated list is actually more accurate for this specific task. The complexity of spaCy would add overhead without improving results at this scope.

Decision 3: In-memory PDF streaming

The report generator writes to an io.BytesIO buffer and streams it directly as a FastAPI StreamingResponse — no file is saved to disk. This avoids file management, works on any deployment without persistent storage, and eliminates cleanup logic.

Decision 4: Streamlit → React migration

Streamlit has hard limits — no custom animations, poor layout control, no real component architecture, and no drag-drop. For a polished project demo, UI quality matters. React gives full control over every interaction, and Tailwind makes it fast to build without writing raw CSS.

PART 3 — SKILLS THIS PROJECT DEMONSTRATES

Use this when an interviewer asks 'what did you learn from this project' or 'what skills does this show?'

Skill Area	How This Project Proves It
Python	All backend modules, NLP logic, PDF processing, report generation written in Python
FastAPI	Built async REST API with 3 endpoints, CORS middleware, file upload, streaming response
React	Full frontend with useState, component props, conditional rendering, useCallback
REST API Design	Designed multipart/form-data endpoints with proper HTTP methods and JSON responses
NLP Fundamentals	Keyword extraction, text preprocessing, information retrieval from unstructured text
System Design	Modular 4-layer architecture — UI, API, Processing, Output — with clean separation
Full-Stack	Owned complete stack from PDF parsing → NLP → API → React UI → PDF report
Problem Solving	Identified Streamlit limitations, migrated to better architecture mid-project
Git	Version controlled entire project, meaningful commits, public GitHub repo
Documentation	README, requirements.txt, inline code comments, API docs via Swagger

PART 4 — INTERVIEW Q&A (Your Version)

These are the questions YOU will be asked in internship interviews about this project.

Story & Context Questions

Q: Tell me about a project you built.

A: I built an AI Resume Analyzer — a full-stack NLP application that analyzes resumes for ATS compatibility. It accepts PDF resumes, extracts text using pdfplumber, detects 40+ technical skills through keyword matching, calculates an ATS score, compares the resume against job descriptions, and generates a downloadable PDF report. The backend is FastAPI in Python and the frontend is React with Tailwind CSS and Recharts for visualization.

Q: Why did you build this project?

A: I built it to help a friend who needed a BCA final year project. The problem it solves is real — most fresh graduates don't know their resumes are being filtered by ATS systems before any human reads them. I wanted to build something with real utility, not a toy project. It also gave me hands-on experience with the full stack from NLP to React UI.

Q: What was the hardest part of building this?

A: The migration from Streamlit to React + FastAPI was the most complex decision. I had a working Streamlit prototype, but Streamlit's limitations in UI control and component architecture made it unsuitable for a polished final product. Restructuring the codebase — separating concerns into FastAPI routes and React components — while keeping all the NLP logic intact required careful planning.

Technical Deep-Dive Questions

Q: Walk me through your FastAPI backend architecture.

A: The backend has main.py with two core endpoints. POST /analyze accepts a multipart/form-data request with the PDF file and optional JD text. It reads the file into a BytesIO buffer, passes it through 6 processing modules, and returns a JSON response. POST /download-report runs the same pipeline but generates a PDF using ReportLab and returns it as a StreamingResponse. CORS middleware allows the React frontend on port 5173 to communicate with FastAPI on port 8000.

Q: How did you handle the PDF file upload in FastAPI?

A: FastAPI's UploadFile type handles multipart/form-data uploads natively. I read the file contents with await file.read(), wrap them in io.BytesIO to create a file-like object in memory, and pass that to pdfplumber. This avoids saving the file to disk entirely — everything stays in memory, which is faster and cleaner for a stateless API.

Q: How does your NLP pipeline work end to end?

A: Step 1: pdfplumber opens the BytesIO object and extracts text page by page. Step 2: The skill detector converts the text to lowercase and checks if each of 40+ skill keywords appears in the text. Step 3: The scorer calculates detected/total times 100. Step 4: The suggestion engine checks the score against thresholds and generates rule-based feedback. Step 5: The section checker looks for Education, Experience, Skills, Projects keywords. Step 6: If JD text is provided, the JD matcher first finds which skills appear in the JD, then checks how many of those are in the resume.

Q: How did you structure your React frontend?

A: The App component manages all state — file, JD text, loading, results, error. It passes handlers down to UploadSection which handles drag-drop via react-dropzone, JD textarea, and the Analyze button with loading spinner. When the API call succeeds, results are passed to ResultsDashboard which renders two RadialBarChart gauges from Recharts, detected/missing skills as colored chip spans, section checker, and suggestions. All styled with Tailwind utility classes.

Q: What is CORS and how did you handle it?

A: CORS is Cross-Origin Resource Sharing — a browser security policy that blocks JavaScript from making requests to a different domain or port than the serving page. React on port 5173 calling FastAPI on port 8000 triggers this. I added FastAPI's CORSMiddleware with allow_origins set to the React dev server URL, allowing all methods and headers. In production, this would be restricted to the deployed frontend domain.

Q: How does the PDF report generation work?

A: The report_generator.py module uses ReportLab's platypus library. I create a SimpleDocTemplate writing to an io.BytesIO buffer instead of a file path. I build a story list of ReportLab flowables — Paragraphs, HRFlowables, Spacers — styled with ParagraphStyle objects. doc.build(story) generates the PDF into the buffer. I call buffer.seek(0) and return the buffer wrapped in FastAPI's StreamingResponse with Content-Disposition header set to attachment, which triggers the browser download.

Architecture & Design Questions

Q: Why did you separate the utils into individual files instead of one file?

A: Single responsibility principle — each module does exactly one thing. extractor.py only extracts text. scorer.py only calculates scores. This makes each module independently testable, easy to modify without breaking other modules, and easy to explain. When we expand to spaCy or ML models, we only replace skill_detector.py — nothing else changes.

Q: How would you scale this system for 10,000 users?

A: I would add a task queue like Celery with Redis to handle PDF processing asynchronously — heavy NLP work should not block the API thread. I would containerize

with Docker and orchestrate with Kubernetes for horizontal scaling. Add a PostgreSQL database for caching results by file hash so identical resumes are not reprocessed. Use a CDN for the React frontend. Move PDF storage to S3 for persistent file access if needed.

Q: What would you add to make this production-ready?

A: User authentication with JWT tokens, PostgreSQL for storing analysis history per user, rate limiting to prevent abuse, input validation and file size limits, proper error logging with structured logs, environment variables for configuration, Dockerfile for containerization, CI/CD pipeline on GitHub Actions, and HTTPS with a proper domain.

PART 5 — HOW TO POSITION THIS PROJECT

Use these descriptions depending on what role you are targeting.

For ML/AI Internship Applications

How to describe it:

I built an NLP-based document analysis pipeline that extracts structured information from unstructured PDF text. The system implements keyword-based named entity extraction for skill detection, a scoring algorithm for ATS compatibility measurement, and a rule-based inference engine for suggestion generation — all served through a FastAPI REST API with a React frontend.

For Full-Stack Internship Applications

How to describe it:

I built a full-stack web application with a FastAPI Python backend and React frontend. The backend processes PDF file uploads, runs NLP analysis across 6 modular Python functions, and returns structured JSON. The React frontend uses hooks for state management, Recharts for data visualization, react-dropzone for file upload, and Axios for API calls. PDF reports are generated with ReportLab and streamed to the browser.

For General SWE Applications

How to describe it:

I built and shipped a complete web application — from initial Streamlit prototype through to a production-grade React + FastAPI architecture. I made key technical decisions including framework selection, API design, modular code architecture, and performance optimization. The project is on GitHub with documentation, setup instructions, and a demo.

Talking Points That Stand Out

- You started with a prototype and iterated to a better architecture — shows engineering judgment, not just execution
- You chose FastAPI over Flask for valid technical reasons — shows you research tools, not just use what you know
- In-memory PDF streaming — shows you think about production concerns like disk I/O and stateless APIs
- Modular architecture — shows you understand separation of concerns and maintainability
- You can explain every line of code — this is your project, you built it, own it

PART 6 — QUICK REFERENCE CHEAT SHEET

Question	Your Answer
What is the project?	Full-stack NLP web app — resume analysis, ATS scoring, JD matching, PDF report
Backend	FastAPI + Python + pdfplumber + ReportLab + Uvicorn
Frontend	React (Vite) + Tailwind CSS + Recharts + Axios + React Dropzone
NLP technique	Keyword-based case-insensitive skill extraction from PDF text
Skills in list	40+ across languages, frameworks, ML/AI, cloud, databases, DevOps
ATS formula	$(\text{Detected Skills} \div \text{Total Skills}) \times 100$
JD Matching	Find JD skills → check which are in resume → score on JD skills only
API endpoints	GET / POST /analyze POST /download-report
Why FastAPI?	Async, fast, auto-docs, type hints — modern Python API standard
Why pdfplumber?	More reliable extraction than PyPDF2 for formatted resumes
Why React?	Component architecture, state management, full UI control vs Streamlit
Why in-memory streaming?	No disk I/O, stateless, works on any deployment, instant response
Hardest part?	Streamlit → React + FastAPI migration while keeping NLP logic intact
Future improvements?	spaCy NER, user auth, PostgreSQL, LLM feedback, Docker, CI/CD
What does it prove?	Python, FastAPI, React, REST API design, NLP, system design, full-stack